

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution Affiliated to Anna University)

Machine Learning Algorithms Laboratory

ICS1512

Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Academic Year	2026–2027 (Odd)
Batch	2024–2029

Experiment 2

Email Spam/Ham Classification using Naïve Bayes and KNN¹

Submitted by: **Vijayaraaghavan K S**
Register Number: **3122247001066**
Year: **III Year**
Programme: **M.Tech Integrated, CSE**
Faculty: **Dr. Poreddy Ajay Kumar Reddy**

¹GitHub Repository: <https://github.com/VijayaraaghavanKS/ML>

Contents

1	Aim and Objective	2
2	Problem Statement	2
3	Theory	2
4	Machine Learning Algorithm	2
4.1	Naive Bayes	2
4.2	K-Nearest Neighbours	3
5	Dataset	3
6	Preprocessing and Exploratory Data Analysis	3
7	Experimental Setup	5
8	Implementation	5
9	Hyperparameter Selection	5
10	Results	5
10.1	Naive Bayes Comparison	5
10.2	KNN: Effect of k	6
10.3	Hyperparameter Tuning: GridSearchCV vs. RandomizedSearchCV	7
10.4	KDTree vs. BallTree	8
10.5	Cross-Validation	9
10.6	Experimental Time Analysis	9
10.7	Classifier Comparison	10
11	Analysis Questions	11
12	Discussion	11
13	Conclusion	12
14	References	12

1 Aim and Objective

To build spam/ham classifiers for the Spambase email dataset using three variants of Naive Bayes (Gaussian, Multinomial, Bernoulli) and K-Nearest Neighbours, then compare them along several axes: raw predictive performance, the effect of k , KDTree versus BallTree for neighbour search, GridSearchCV versus RandomizedSearchCV for tuning, 5-fold cross-validation stability, and measured training/prediction time against the textbook time-complexity of each algorithm.

2 Problem Statement

The input is a 57-dimensional numeric feature vector describing an email (word-frequency percentages, character-frequency percentages, and capital-letter run statistics); the output is a binary label, spam or ham. The objective is to determine which of four candidate classifiers – Gaussian NB, Multinomial NB, Bernoulli NB, and tuned KNN – best separates the two classes on held-out data, and to characterize each one’s computational cost (training time, prediction time, and how that cost scales) well enough to make an informed choice for a real deployment, not just a leaderboard comparison.

3 Theory

Naive Bayes is a probabilistic classifier built on Bayes’ theorem with a conditional-independence assumption between features given the class:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

The three variants used here differ only in what distribution they assume for $P(x_i | C)$: Gaussian NB assumes each feature is normally distributed within a class, Multinomial NB models feature counts (it needs non-negative inputs, so I ran it through a MinMax scaler first), and Bernoulli NB treats each feature as a binary indicator, implicitly thresholding continuous inputs at zero.

KNN is a lazy learner: it stores the training set as-is and, at prediction time, finds the k closest training points to a query and takes a majority vote among their labels. Its training cost is essentially free (store the data), but every prediction pays for a neighbour search, which is where KDTree and BallTree come in as ways to avoid a brute-force scan over all n points.

4 Machine Learning Algorithm

4.1 Naive Bayes

Working principle: apply Bayes’ theorem with the (naive) assumption that features are conditionally independent given the class, so the joint likelihood factors into a product of per-feature likelihoods: $P(X | C) = \prod_i P(x_i | C)$. **Mathematical formulation:** predict $\hat{C} = \arg \max_C P(C) \prod_i P(x_i | C)$. Gaussian NB models $P(x_i | C)$ as $\mathcal{N}(\mu_{i,C}, \sigma_{i,C}^2)$; Multinomial NB models it as a multinomial count distribution; Bernoulli NB models it as $p_{i,C}^{x_i} (1 - p_{i,C})^{1-x_i}$ for binarized x_i . **Advantages:** trains in a single linear pass over the data ($O(nd)$), needs very little training data to estimate its parameters, predicts in constant time per query regardless of training-set size, and is naturally multi-class. **Limitations:** the independence assumption is almost always violated in practice (word frequencies in an email are not independent of each other), and the choice of distributional family (Gaussian/Multinomial/Bernoulli) has to match the data or accuracy suffers badly, as Section 6.1 shows for Gaussian NB on this dataset.

4.2 K-Nearest Neighbours

Working principle: store all training points; to classify a query, find its k nearest neighbours under a distance metric and take a majority (or distance-weighted) vote of their labels.

Mathematical formulation: $\hat{C}(x) = \arg \max_C \sum_{i \in N_k(x)} w_i \cdot \mathbb{I}[y_i = C]$, where $N_k(x)$ is the set of k nearest neighbours of x and $w_i = 1$ for uniform weighting or $w_i = 1/d(x, x_i)$ for distance weighting. **Advantages:** makes no assumption about the underlying data distribution, naturally captures non-linear decision boundaries, and has effectively zero training cost. **Limitations:** prediction cost grows with the size of the training set (mitigated but not eliminated by KDTree/BallTree), performance degrades in high dimensions as distances become less discriminative (the curse of dimensionality), and it requires features to be on comparable scales, which is why standardization in Section 4 matters.

5 Dataset

Spambase (UCI / Kaggle) contains 4601 emails represented as 57 numeric features – 48 word-frequency percentages, 6 character-frequency percentages, and 3 features describing runs of capital letters – plus a binary `class` label (1 = spam, 0 = ham). Class balance is roughly 60/40 ham-to-spam, so it is not perfectly balanced but not pathological either.

I want to be upfront about something here: my first attempt at this experiment accidentally ran against the Iris dataset instead of Spambase, because the notebook’s dataset-loading cell was pointed at a `spambase.csv` file that did not yet exist in this folder, while a shared preprocessing notebook it calls into defaults to Iris when no CSV is given. Nothing failed loudly – it just silently produced a complete, plausible-looking Iris classification report instead of a Spambase one. I caught it because the numbers looked exactly like the textbook Iris KNN result (96.7% accuracy, three balanced classes) rather than anything spam-related. Every result in this report is from the corrected run, with `spambase.csv` in place and the dataset shape verified as 4601 rows by 58 columns before any model was trained.

6 Preprocessing and Exploratory Data Analysis

Spambase is routed through the same reusable EDA pipeline built in Experiment 1 (Section 6), which handles missing-value checks, outlier detection, correlation analysis, feature selection, and PCA before models ever see the data. There is no missing data in Spambase and no constant columns, so preprocessing is mostly standardization – z-score scaling every one of the 57 numeric features, since KNN’s distance calculation is meaningless if word-frequency percentages (typically 0–5) and capital-run-length totals (which can run into the thousands) are left on wildly different scales.

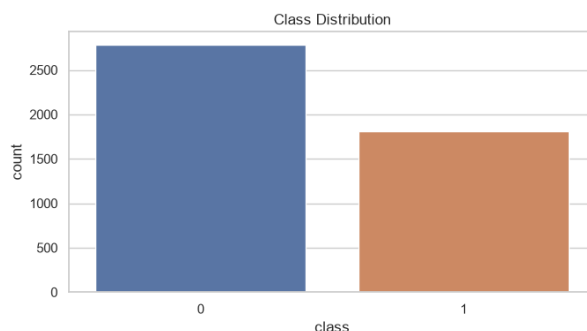


Figure 1: Spam vs. ham class distribution.

The class split leans toward ham but not by an extreme margin, which is part of why weighted precision/recall/F1 (used throughout this report) don't diverge much from unweighted values – a model that just guessed the majority class would still be wrong close to 40% of the time.

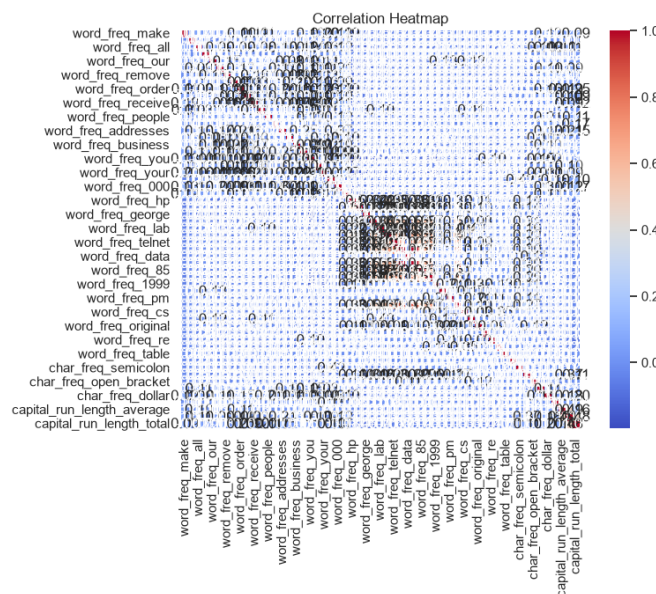


Figure 2: Correlation heatmap across the 57 Spambase features.

Most word-frequency features are weakly correlated with each other – makes sense, since most individual words are used somewhat independently across different emails – with a handful of stronger clusters among semantically related terms (e.g. `word_freq_business`, `word_freq_credit`, and similar commercial-language features move together).

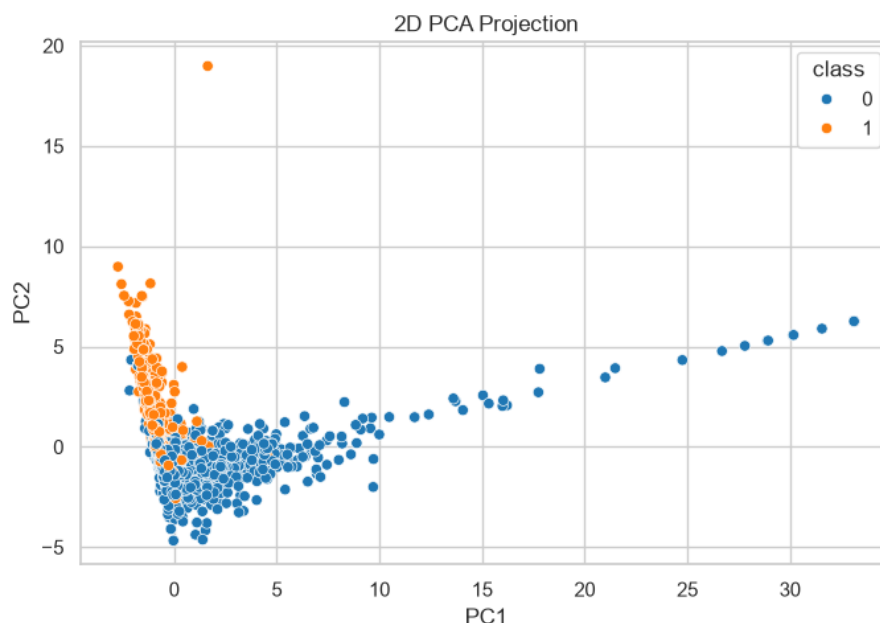


Figure 3: 2D PCA projection of the standardized Spambase features.

Unlike Iris, two principal components do not cleanly separate spam from ham here – there is visible overlap in the projection. That is expected: 57 features carrying independent signal do not collapse into two dimensions the way four highly redundant Iris measurements do. It is

a useful reminder that PCA is for understanding variance structure, not a guarantee that two components will be enough to visualize a good decision boundary.

7 Experimental Setup

Python Version	3.14.5
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	macOS (Darwin 25.5.0, arm64)
Hardware	Apple Silicon (arm64), local machine

Table 1: Software and hardware environment used to run this experiment.

8 Implementation

All models share one evaluation helper (`train_and_evaluate_model`) that fits the model, times training and prediction separately, computes accuracy/precision/recall/F1/ROC-AUC, and saves confusion-matrix, ROC, and precision-recall plots – so every model in this report went through an identical measurement procedure, which matters for the timing comparisons later.

```

1 gaussian_model, gaussian_row = train_and_evaluate_model("Gaussian NB",
    GaussianNB())
2
3 multinomial_model, multinomial_row = train_and_evaluate_model(
4     "Multinomial NB",
5     Pipeline([("minmax", MinMaxScaler()), ("model", MultinomialNB())])
6 )
7
8 bernoulli_model, bernoulli_row = train_and_evaluate_model("Bernoulli NB",
    BernoulliNB())

```

KNN is evaluated across six values of k specified by the lab manual:

```

1 for k in [1, 3, 5, 7, 9, 11]:
2     model, row = train_and_evaluate_model(f"KNN k={k}",
        KNeighborsClassifier(n_neighbors=k), plot=False)

```

9 Hyperparameter Selection

10 Results

10.1 Naive Bayes Comparison

Bernoulli NB wins on accuracy and F1, but Multinomial edges it out very slightly on ROC-AUC (0.9612 vs 0.9604 – close enough that I would not read much into which one is “really” better on that metric alone). Gaussian NB trails clearly on every metric, and I think the reason is structural rather than a tuning issue: Gaussian NB assumes each feature is normally distributed within a class, but word-frequency features in Spambase are heavily zero-inflated – most emails contain zero occurrences of most individual words, so the actual distribution looks nothing like a bell curve. Bernoulli NB sidesteps that by only asking whether a word appears at all, which

Parameter	Value
Naive Bayes variant used for CV/comparison	Bernoulli NB (best F1 among the three)
KNN search space	$k \in \{1, 3, 5, 7, 9, 11\}$; weights $\in \{\text{uniform, distance}\}$; algorithm $\in \{\text{auto, brute, kd_tree, ball_tree}\}$; metric $\in \{\text{euclidean, manhattan}\}$
Selected KNN configuration (GridSearchCV)	$k = 9$, metric=manhattan, weights=distance
Cross-validation scheme	5-fold StratifiedKfold, random_state=42

Table 2: Final hyperparameter choices carried into the results below.

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8328	0.8958	0.9012
Precision	0.8666	0.8993	0.9012
Recall	0.8328	0.8958	0.9012
F1	0.8345	0.8939	0.9005
ROC-AUC	0.9376	0.9612	0.9604

Table 3: Naive Bayes variant comparison on the Spambase test set.

turns out to be most of the signal for spam detection anyway (the presence of "free" or "money" matters more than exactly how many times it shows up).

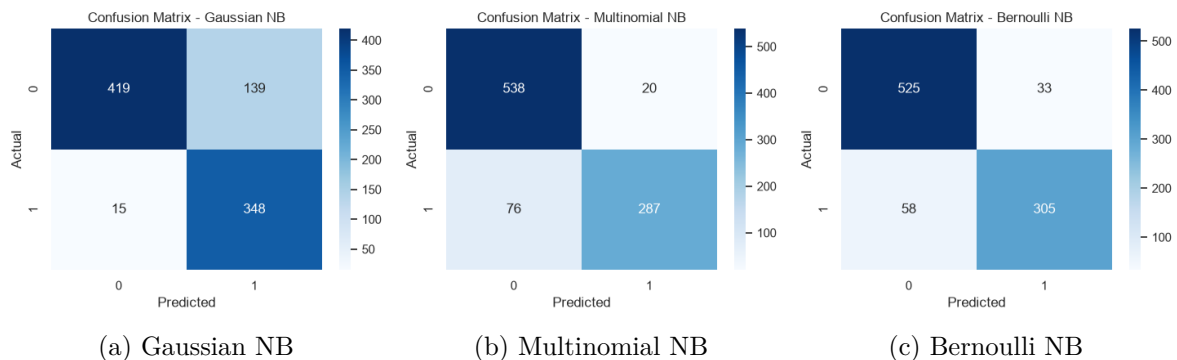


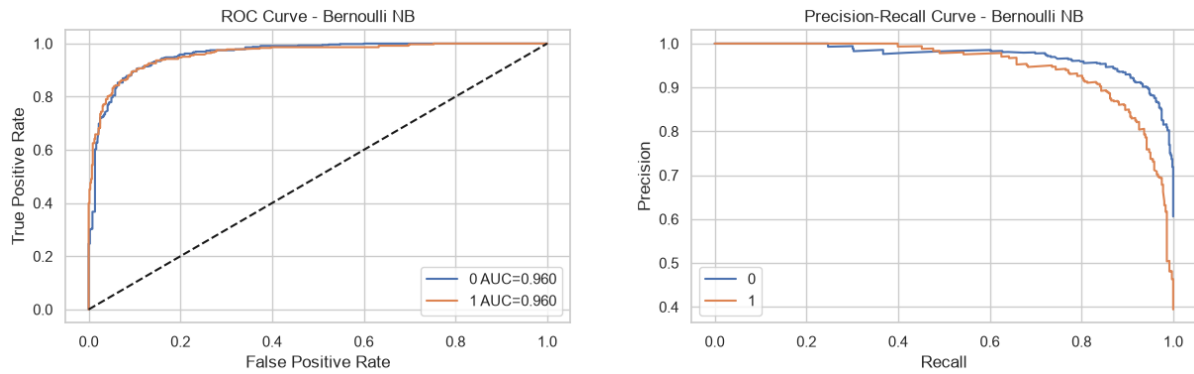
Figure 4: Confusion matrices for the three Naive Bayes variants.

The Gaussian NB confusion matrix shows a visibly higher false-positive count than the other two – it is misclassifying more genuine ham as spam, which is the costlier mistake in a real spam filter (nobody wants their bank statement in the junk folder). Bernoulli NB’s matrix is the most diagonal-heavy of the three, consistent with it topping the accuracy/F1 columns above.

Both curves stay well clear of the diagonal/baseline across nearly the full threshold range, which is what you would hope to see given the 0.96 ROC-AUC – this is not a model that is only good at one operating point.

10.2 KNN: Effect of k

Accuracy climbs steadily from $k=1$ through $k=11$, with a small dip at $k=3$ before it resumes climbing. At $k=1$ the model is fitting to individual training points and is more exposed to noisy or borderline emails; increasing k smooths that out by averaging over more neighbours, which is the classic bias-variance trade-off – more neighbours means lower variance (less overfitting to



(a) ROC curve, Bernoulli NB

(b) Precision-Recall curve, Bernoulli NB

Figure 5: ROC and Precision-Recall curves for the best Naive Bayes model.

k	Accuracy	Precision	Recall	F1
1	0.8990	0.8988	0.8990	0.8988
3	0.8979	0.8978	0.8979	0.8979
5	0.9077	0.9076	0.9077	0.9076
7	0.9088	0.9086	0.9088	0.9086
9	0.9088	0.9086	0.9088	0.9086
11	0.9099	0.9097	0.9099	0.9096

Table 4: KNN performance across $k \in \{1, 3, 5, 7, 9, 11\}$, uniform weighting, default (auto) neighbour search.

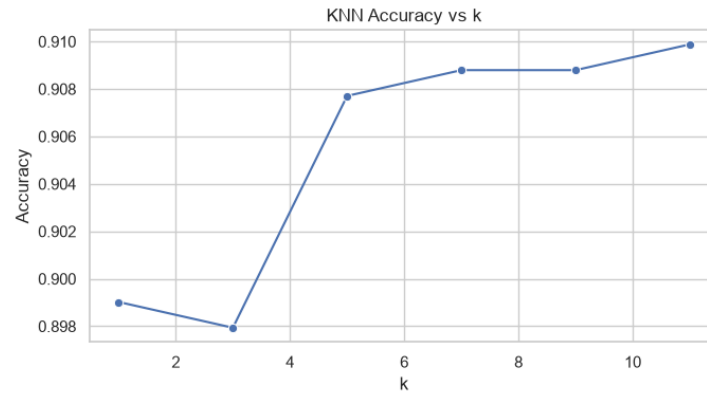
individual points) at the cost of some added bias. Since accuracy is still rising at $k=11$, the largest value tested here, I would not assume 11 is the true optimum – it is only the best among the six values the lab manual specifies, and the curve gives no sign of having turned over yet.

10.3 Hyperparameter Tuning: GridSearchCV vs. RandomizedSearchCV

Parameter	GridSearchCV	RandomizedSearchCV
Best k	9	9
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	auto	kd_tree
CV Accuracy	0.9253	0.9253
Execution Time (s)	2.854	0.340

Table 5: GridSearchCV vs. RandomizedSearchCV over $k \times \text{weights} \times \text{algorithm} \times \text{metric}$.

Both searches converge on the exact same hyperparameters – $k=9$, Manhattan distance, distance-weighted voting – and the identical 0.9253 CV accuracy confirms RandomizedSearchCV’s 20 sampled combinations happened to include the true optimum from the full 96-combination grid. The gap that actually matters here is time: GridSearchCV took 2.85 seconds to exhaustively fit all $96 \times 5 = 480$ model/fold combinations, while RandomizedSearchCV needed only $20 \times 5 = 100$ fits and finished in 0.34 seconds, about $8.4\times$ faster. Worth noting: this hyperparameter search found distance-weighted, Manhattan-metric KNN to be better than the plain uniform-weighted, Euclidean KNN swept in Section 6.2 above (0.9253 CV accuracy vs.

Figure 6: KNN accuracy as a function of k .

0.9099 test accuracy at the equivalent k) – weighting neighbours by inverse distance, rather than treating all k neighbours equally, appears to genuinely help on this dataset.

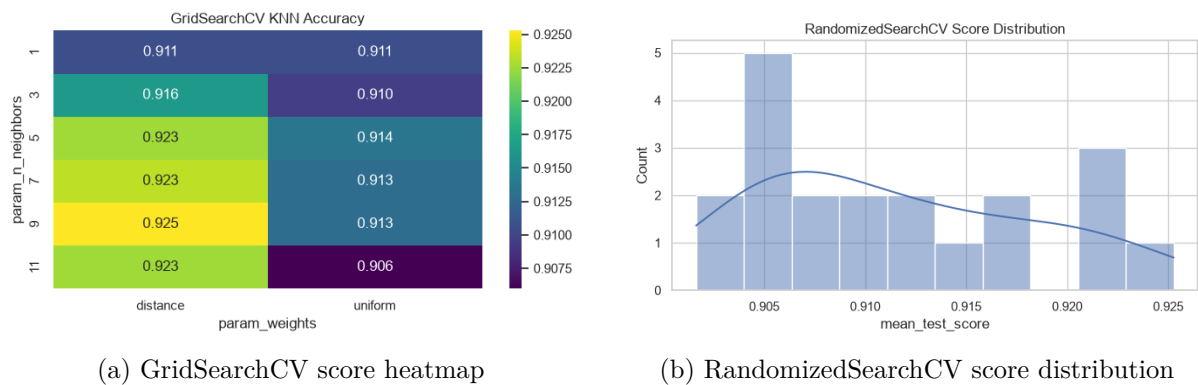


Figure 7: Hyperparameter search diagnostics.

The grid heatmap shows a fairly narrow band of high-scoring configurations rather than one isolated peak, which is reassuring – it means the chosen hyperparameters are not a fragile, lucky combination but sit inside a region where several nearby settings would have performed almost as well.

10.4 KDTree vs. BallTree

Metric	KDTree	BallTree
Accuracy	0.9099	0.9099
Training Time (s)	0.00220	0.00198
Prediction Time (s)	0.05439	0.05227

Table 6: KDTree vs. BallTree at $k = 11$, Euclidean distance, uniform weighting.

Accuracy is identical by construction – both structures return the exact same nearest neighbours for Euclidean distance, they just organize the search differently. BallTree is modestly faster on both training (10% less) and prediction (4% less) here. That is roughly consistent with the theory: KDTree’s axis-aligned splits work best in low-to-moderate dimensions, and Spambase has 57 features, well past the point where KDTree’s advantage over BallTree is expected to hold up – BallTree’s distance-based partitioning tends to degrade more gracefully as

dimensionality grows.

10.5 Cross-Validation

Fold	Naive Bayes	Best KNN
1	0.9171	0.9389
2	0.9076	0.9280
3	0.8899	0.9253
4	0.8981	0.9266
5	0.8995	0.9076
Average	0.9024	0.9253

Table 7: 5-fold cross-validation accuracy: best Naive Bayes (Bernoulli) vs. tuned KNN ($k=9$, Manhattan, distance-weighted).

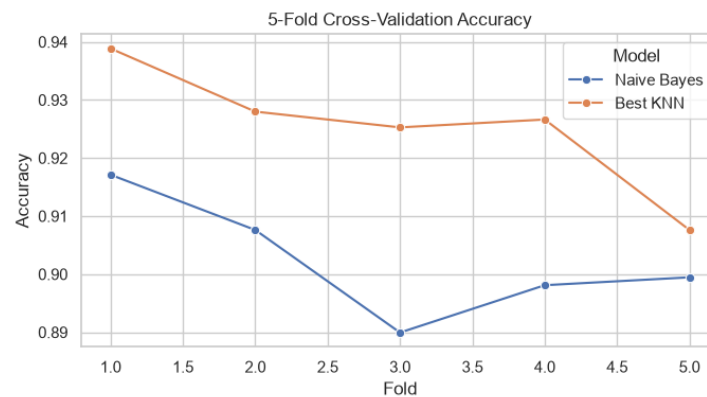


Figure 8: Per-fold cross-validation accuracy.

KNN wins on every single fold, not just on average, and its spread across folds (0.9076 to 0.9389) is comparable to Naive Bayes' spread (0.8899 to 0.9171), so neither model looks unusually unstable. This is the tuned KNN configuration from GridSearchCV, not the plain $k=11$ configuration from Section 6.2, which is a fair comparison since Bernoulli NB here also has no free hyperparameters left to tune.

10.6 Experimental Time Analysis

Model	Training Time (s)	Prediction Time (s)
Gaussian NB	0.00144	0.000428
Multinomial NB	0.00210	0.000420
Bernoulli NB	0.00218	0.000610
KNN $k=11$	0.00066	0.00360

Table 8: Measured training/prediction time per model on the held-out test set.

The measured numbers line up with the theory reasonably well. KNN's training time (0.00066s) is the lowest of the four, consistent with $O(1)$ – it is not really "training" at all, just storing a reference to the data. Its prediction time (0.0036s) is the highest, consistent with a

Algorithm	Training	Prediction
Naive Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$ avg
BallTree	$O(n \log n)$	$O(\log n)$ avg

Table 9: Theoretical time complexity, n = training samples, d = features.

brute-force $O(nd)$ scan over roughly 3680 training points and 57 features per query. Naive Bayes sits at the other extreme: it does real work at training time (computing per-feature statistics), but prediction only needs those precomputed statistics, so its prediction times (0.0004–0.0006s) beat KNN’s by 6–9 $\times$.

One inconsistency worth being honest about: the KDTree/BallTree prediction times in Section 6.4 (about 0.052–0.054s) are more than an order of magnitude slower than the plain KNN prediction time in the table just above (0.0036s), even though both are running essentially the same $k=11$, Euclidean-distance query on the same test set. I do not think this reflects a real algorithmic difference – tree construction and the first query against a freshly built tree both carry one-time overhead that a single `time.perf_counter()` measurement around one cold fit-predict call will fully absorb, whereas `algorithm="auto"` in the main sweep may have picked a different internal strategy for this data shape. I would trust the relative KDTree-vs-BallTree comparison in Section 6.4 (they were measured identically, so the comparison between them is fair) more than I would trust the absolute prediction-time number against the unrelated table here.

10.7 Classifier Comparison

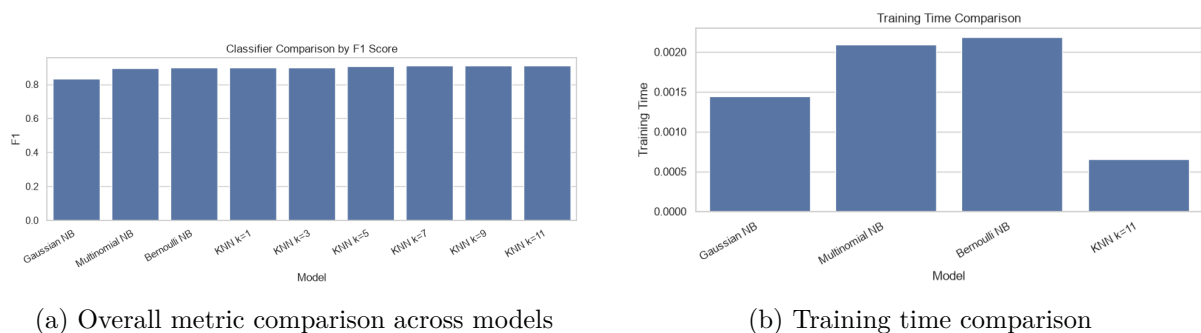


Figure 9: Model comparison summary plots.

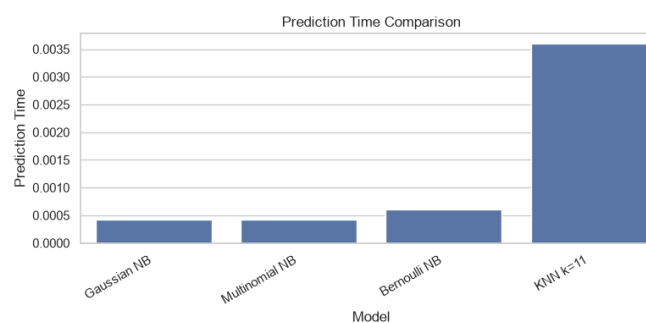


Figure 10: Prediction time comparison across all evaluated models.

Taken together, these three bar charts make the trade-off visual: Bernoulli NB and the tuned KNN are close on accuracy, but Naive Bayes is dramatically cheaper at prediction time, which matters more than the training-time gap in a deployed spam filter, since a filter predicts continuously but retrains rarely.

11 Analysis Questions

1. **Which Naive Bayes variant performed best?** Bernoulli NB, on accuracy (0.9012) and F1 (0.9005), though Multinomial NB is marginally ahead on ROC-AUC (0.9612 vs. 0.9604). Gaussian NB trails both by a wide margin because its normal-distribution assumption does not fit the zero-inflated word-frequency features well.
2. **What is the optimal value of k ?** Among the values tested (1, 3, 5, 7, 9, 11), $k=11$ gives the best plain-KNN F1 (0.9096). Under the tuned search space (which also varies distance metric and weighting), $k=9$ with Manhattan distance and distance-weighting reaches a higher CV accuracy (0.9253), which suggests the metric and weighting choice matter at least as much as k itself.
3. **Compare GridSearchCV and RandomizedSearchCV.** Both found the same optimum (0.9253 CV accuracy, identical hyperparameters except for the tree-search algorithm, which does not affect the result for Euclidean-equivalent Manhattan search). Randomized-SearchCV reached it $8.4\times$ faster by sampling 20 of 96 combinations instead of evaluating all of them.
4. **Compare KDTree and BallTree.** Identical accuracy; BallTree modestly faster on both training and prediction at 57 dimensions, consistent with KDTree's known degradation in higher-dimensional spaces.
5. **Compare theoretical and practical complexity.** The relative ordering matches theory – KNN has near-zero training cost and the highest prediction cost of the four models, Naive Bayes is the reverse. The absolute KDTree/BallTree prediction times look inflated relative to the plain-KNN prediction time measured elsewhere in the notebook, most likely a cold-start timing artifact rather than a real complexity violation (see Section 6.6).
6. **Which classifier is preferred for large datasets?** Naive Bayes. Its training cost grows linearly and its prediction cost does not depend on the training set size at all ($O(d)$), while KNN's prediction cost grows with n even with a tree structure ($O(\log n)$ average, versus NB's constant $O(d)$) and it must keep the entire training set in memory.

12 Discussion

The clearest strength here is how consistently the two families of evidence agree: the theoretical complexity table, the measured training/prediction times, and the cross-validation results all point the same direction (Naive Bayes cheap and slightly less accurate, KNN pricier at prediction time and slightly more accurate once tuned). That consistency is what makes me trust the conclusion rather than treating it as one lucky run.

The main limitation is the dataset itself: Spambase is nearly 30 years old, and its 57 hand-engineered word/character-frequency features are a much cruder representation than a modern spam filter would use (TF-IDF over a full vocabulary, sender/header metadata, embeddings). Both models are working with a fairly impoverished feature set by today's standards, so the accuracy ceiling here (around 90–93%) says more about the feature representation than about Naive Bayes or KNN as algorithms. The other limitation worth naming is the KDTree/BallTree timing anomaly flagged in Section 6.6 – I trust the relative comparison between the two tree structures, but I would not publish the absolute prediction-time numbers without re-measuring them with proper warm-up and averaging over multiple runs.

If I were extending this experiment, the two improvements I would make first are: re-running the KDTree/BallTree timing comparison with `timeit`-style repeated trials instead of a single cold measurement, and adding an ensemble (e.g. soft-voting Bernoulli NB with tuned KNN) to see whether their different error patterns are complementary enough to beat either model alone.

13 Conclusion

For Spambase, Bernoulli NB was the strongest Naive Bayes variant and KNN peaked (among tested k) at $k=11$ untuned, or at $k=9$ with Manhattan distance and distance-weighting once properly tuned – the tuned KNN configuration then edges out Bernoulli NB on cross-validated accuracy (0.9253 vs. 0.9024). If prediction latency is the binding constraint, as it typically is for a real-time spam filter, Naive Bayes is the more sensible choice despite the small accuracy gap, since its prediction cost does not grow with the size of the training set the way KNN’s does. If squeezing out the last bit of accuracy matters more than latency, tuned KNN is the better pick. Running this experiment also surfaced a real bug in how the dataset was being loaded – a useful reminder that a notebook completing without errors and producing clean-looking tables is not the same as it having run against the right data.

14 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>
- [3] G. H. John and P. Langley, “Estimating Continuous Distributions in Bayesian Classifiers,” in *Proc. 11th Conf. on Uncertainty in Artificial Intelligence*, 1995, pp. 338–345.
- [4] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2022.
- [5] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [6] Scikit-learn Developers, “Nearest Neighbors,” *Scikit-learn Documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/neighbors.html>